

高云 FPGA 多功能协议调试器

摘要

我们设计了一款基于高云 ACG720-138K 的多功能协议调试器，支持实现下面五个主要功能：通过 USB CDC（USB2.0）驱动的 CDC 转 UART、SPI、IIC、PWM、多路用户自定义多通道循环序列输出；8 通道数字信号采集与协议解析（支持 UART，IIC，SPI）；双通道多种波形（包括四种常见波形和用户自定义手绘波形）模拟信号发生器（支持可调幅度，相位、幅值等）；双通道模拟信号采集和波形显示（支持可调采样率，采样深度等）；支持数字信号测量，能够测量周期性数字信号的频率、占空比、高低电平时间。

目录

1	第一部分 作品概述	4
1.1	功能与特性	4
1.2	应用领域	4
1.3	主要技术特点	5
1.4	主要性能指标	6
1.5	主要创新点	9
1.6	设计流程	10
2	第二部分 系统组成及功能说明	11
2.1	整体介绍	11
2.2	硬件系统介绍	13
2.2.1	硬件整体介绍；	13
2.2.2	电路重要模块及其接口介绍：	14
2.3	上下位机通信详细协议帧：	20
2.3.1	模拟信号发生器	20
2.3.2	PWM 和自定义序列发生器	21
2.3.3	CDC 转 IIC 读写控制器	22
2.3.4	CDC 转 SPI 控制器	23
2.3.5	手绘波形发送器	23
2.3.6	逻辑分析仪	24
2.4	软件系统介绍	25
2.4.1	软件整体介绍（含 PC 端或云端，结合关键图片）；	25
2.4.2	软件各模块介绍	28
2.4.3	流程分析	29
3	第三部分 完成情况及性能参数	36
3.1	整体介绍（整个系统实物的正面、斜 45°全局性照片）	36

3.2	工程成果（分硬件实物、软件界面等设计结果）	36
3.2.1	电路成果；	36
3.2.2	软件成果；	39
3.3	特性成果（逐个展示功能、性能参数等量化指标）	41
4	第四部分 总结	44
4.1	可扩展之处	44
4.2	心得体会	44
5	第五部分 参考文献	46

1 第一部分 作品概述

1.1 功能与特性

我们设计了一款基于高云 ACG720-138K 的多功能协议调试器，支持实现下面五个主要功能：

- (1) 通过 USB CDC (USB2.0) 驱动的 CDC 转 UART、SPI、IIC、PWM (四路)、四路用户自定义多通道循环序列输出；
- (2) 8 通道数字信号采集与协议解析 (支持 UART, IIC, SPI)；
- (3) 双通道多种波形 (包括四种常见波形和用户自定义手绘波形) 模拟信号发生器 (支持可调幅度，相位、幅值等)；
- (4) 双通道模拟信号采集和波形显示 (支持可调采样率，采样深度等)；
- (5) 支持数字信号测量，能够测量周期性数字信号的频率、占空比、高低电平时间并在数码管上显示。

1.2 应用领域

本多功能协议调试器是一款集数字与模拟信号处理于一体的高集成度便携式工具，有很大潜力可以广泛适用于以下领域：

- (1) 嵌入式系统开发与调试：作为核心应用，它可模拟 UART、SPI、I2C 等总线主机，与微控制器(MCU)、传感器及各类外设进行快速通信验证与固件烧录。同时，其数字信号采集与协议解析功能，能高效捕获并分析总线数据，精准定位通信故障。
- (2) 电子产品研发与测试：在产品研发阶段，可利用其双通道信号发生器模拟传感器信号或电源纹波，测试电路的响应特性。双通道模拟采集功能则可

作为简易示波器，观测关键节点的电压波形。PWM 输出可用于电机驱动、LED 调光等场景的快速原型验证。

（3）教育与科研：集成化的设计极大简化了实验平台的搭建。学生和研究人员可以利用它学习和实践串行通信协议、信号处理及数模转换等知识，是电子工程、自动化及物联网等专业理想的教学与实验设备。

（4）现场维护与故障诊断：凭借其便携性和全面的功能，工程师可携带该设备到现场，快速测量数字信号的频率、占空比等关键参数，并对故障设备进行信号级的诊断，显著提升维护效率。

1.3 主要技术特点

本调试器基于高云 ACG720 FPGA 核心，打造了一款高度集成的混合信号测试平台。其核心特点如下：

（1）高度集成与强大核心：利用 FPGA 的并行处理能力，单设备集成了**逻辑分析仪、协议发生器、示波器与信号发生器**等多种仪器功能，实现了硬件资源的极致复用。

（2）智能混合通信架构：采用“控制与数据分离”的先进架构。**USB-CDC 接口**统一管理所有控制命令、协议转换、数字信号与测量结果上传；专用的**以太网接口**则专门负责高速模拟采集数据的无损传输，从根源上避免了总线拥塞，确保系统高效稳定。

（3）软硬件协同工作：**FPGA** 负责高精度的多通道信号实时采集与硬件测量，上位机则专注于复杂的协议解析与丰富显示。这种分工使得协议更新无需改动硬件，极大提升了系统的灵活性和处理能力。

（4）全功能模拟信号处理：双通道信号发生器支持四种标准波形与**手绘波形**，并通过 USB 灵活配置；双通道模拟采集支持高采样率，数据通过以太网实

时传输，实现高质量的波形显示与分析。

（5）智能测量与显示：数字信号频率、占空比、高低电平等参数由 FPGA 硬件实时测量，并通过数码管显示。时间参数单位（ns/μs）可**自适应切换**，并由 LED0 指示灯直观标识（亮=ns，灭=μs），使现场读数快速无歧义。

（6）混合信号联调能力：平台支持模拟/数字信号的同步发生、采集与测量，可轻松构建复杂的测试场景，精准捕捉信号交互问题，实现真正的**混合信号联合调试**。

1.4 主要性能指标

通道数	支持协议	采样率	采样方式	采样深度	测量	传输接口
8 通道同步采集	UART、IIC、SPI	固定 48M Hz	可选触发采样或直接采样；触发采样支持上升沿下降沿双边沿及高低电平	可选 1000 至 50000 个采样点	支持双光标测量，读取时间区间并自动计算频率	USB2.0 CDC

表格 1 数字协议解析性能指标表

CDC 转 IIC	CDC 转 SPI	CDC 转 UART
FPGA 作为 IIC 主机，用户可调控器件地址，寄存器地址，一次最多写入/读取 16 个字节	FPGA 作为 SPI 主机发送（MOSI）读取（MISO）同时进行	CDC 可与 UART 回环传输数据，可调任意波特率、停止位、校验位

表格 2 CDC 转基本协议

模式：	通道数	最小时间精度	最大可调时间	其他特点
CDC 转 PWM	四通道可独立配置参数	1us	1s	支持用户通过调控高电平时间或周期占空比的方式调控参数
CDC 转自定义序列	四通道可独立配置参数	1us	1s	支持最多 24 位用户自定义序列循环输出，用户端只需填入需要的序列

表格 3 CDC 转 PWM 与自定义序列

测量参数			显示方式
频率	高低电平时间	占空比	数码管显示
采用等精度测量法，理论可测频率无限大，实际数码管显示常用频段：1Hz 至 10Mhz	精度为 1%的要求下最大可测 1Mhz，时间单位根据输入信号自适应，LED0 亮表示单位为 ns，LED0 灭表示单位为 us	显示到百分比 小数点后一位，精度较高	显示形式为十进制数，通过 S0 控制显示频率、高低电平时间、占空比

表格 4 数字信号测量

通道数	采样率	波形类型	传输接口	可调参数
双通道独立输出	50Msps	正弦波，方波，三角波，锯齿波及用户自定义手绘波形	USB2.0 CDC	频率（100Hz 至 1MHz 可调）， 相位（0 至 360 度可调）

表格 5 模拟信号发生

通道数	采样率	采样深度	传输接口
双通道独立输入	25M 以下任意可调 (需能被 50M 整除)，可超频至 50M	1K 至 4M 可调	以太网接口

表格 6 模拟信号传输存储与采集

以上表格 1-6 为主要性能指标表。

1.5 主要创新点

（1）混合通信架构：首创 USB-CDC 与以太网的物理分离架构，USB 专注指令控制，以太网专攻高速数据传输，从硬件底层杜绝总线拥塞，实现高效稳定通信。

(2) 硬件智能测量：FPGA 直接实现高精度时间测量与自适应单位切换，并通过 LED 灯光语言（亮/灭）直观显示纳秒/微秒量程，实现测量结果的极低延迟与智能解读。

(3) 高度集成设计：基于单一 FPGA 芯片，高效复用了逻辑分析仪、协议分析仪、双通道示波器及信号发生器等多种仪器功能，实现了真正的单片化混合信号测试平台。

1.6 设计流程

核心设计流程：

(1) 先定义协议：严格制定 USB-CDC 通信协议框架（帧头帧尾即数据帧内容协议等），形成上下位机通用契约文档。

(2) 分层并行实现：

FPGA 端：自底向上构建串口收发器→协议解析状态机（字节流处理/校验）→业务功能模块（信号发生/测量等），关键：通过协议仿真模拟上位机行为预验证。

上位机端：封装异步串口驱动（线程安全缓冲）→事件驱动协议解包模块→核心业务逻辑→独立 UI 界面（强制通信隔离）。

(3) 系统集成验证：

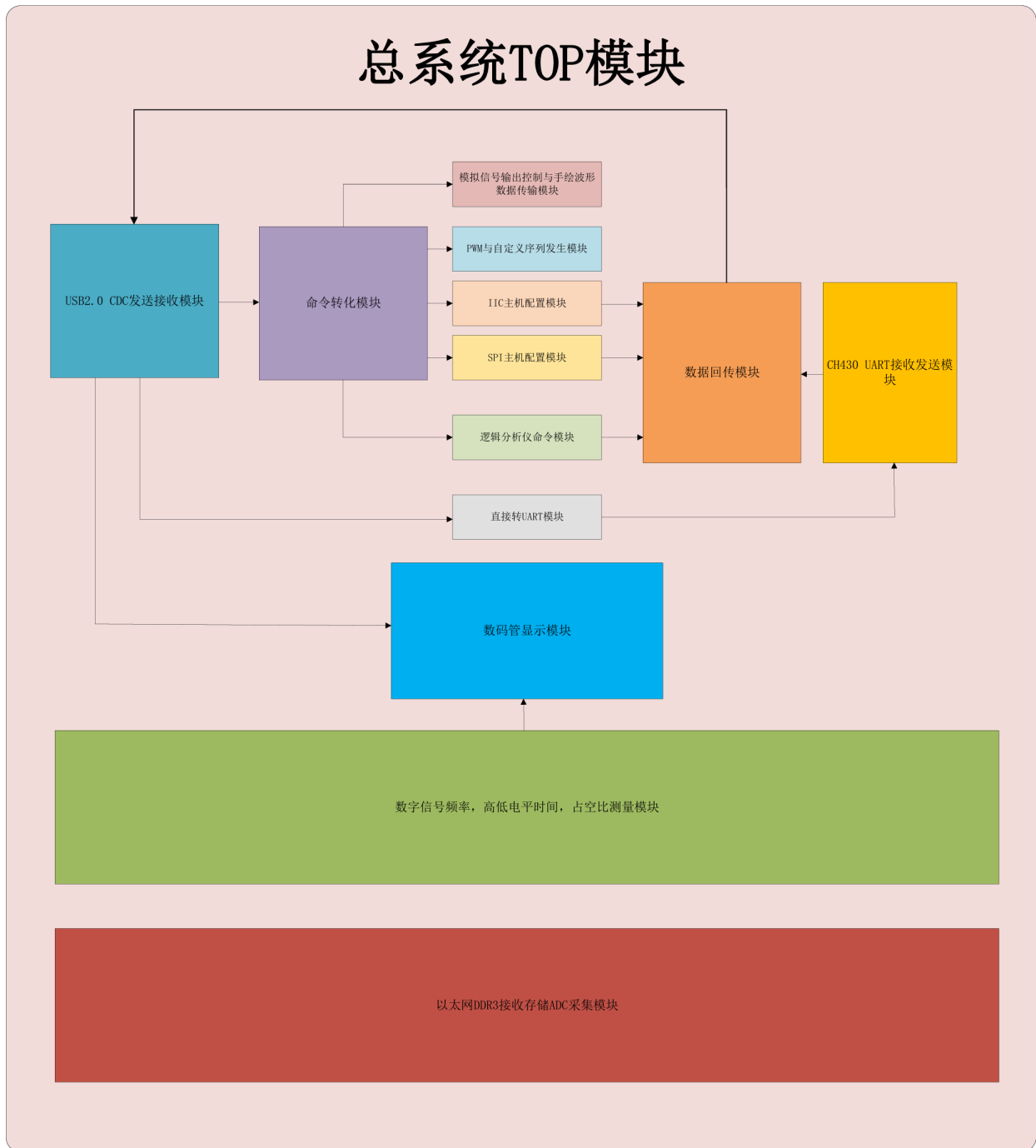
分层联调：先验证基础通信链路，再分模块测试核心功能。

协同诊断：融合上位机原始帧日志、FPGA 在线逻辑分析仪、外接逻辑分析仪与示波器、USB 和以太网抓包工具三方数据定位异常。

2 第二部分 系统组成及功能说明

2.1 整体介绍

系统整体框图如图 1 所示：



图表 1 系统整体模块框图

其中，总模块以通信接口为划分分为三个模块：

(1) 模块一：USB2.0 CDC 接口部分模块，此模块功能涵盖：CDC 转

UART、SPI、IIC、PWM 与用户自定义波形；模拟信号输出及用户自定义波形输出控制；逻辑分析仪数据传输。

(2) 模块二：数字信号测量模块，该模块不通过上位机显示，而是直接在数码管上显示，简单直接便携。

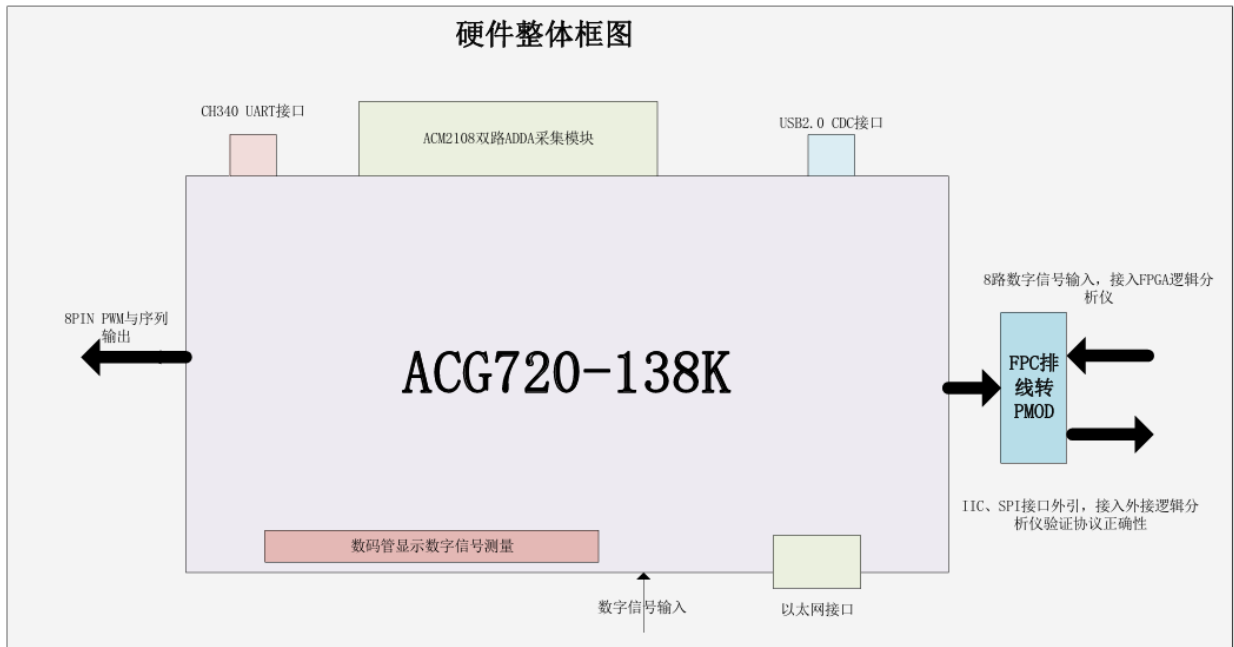
(3) 模块三：DDR3 存储 ADC 采样以太网传输模块，该模块采用千兆以太网接口固定 UDP 协议，能够更加高效稳定传输 ADC 采样数据。

将一个大的系统分成三个子模块，这种做法有效降低了系统设计难度和后期维护优化难度，三个系统之间几乎没有控制线或信号线关联（只有一个数码管需要通过 SW5 滑动开关来控制显示选择）。

2.2 硬件系统介绍

2.2.1 硬件整体介绍：

硬件部分主要由 ACG720-138K FPGA 组成，外挂 ACM2108、FPC 转 PMOD 转接板，通过高速 SMA 线或杜邦线与外界信号交互，如图 2 所

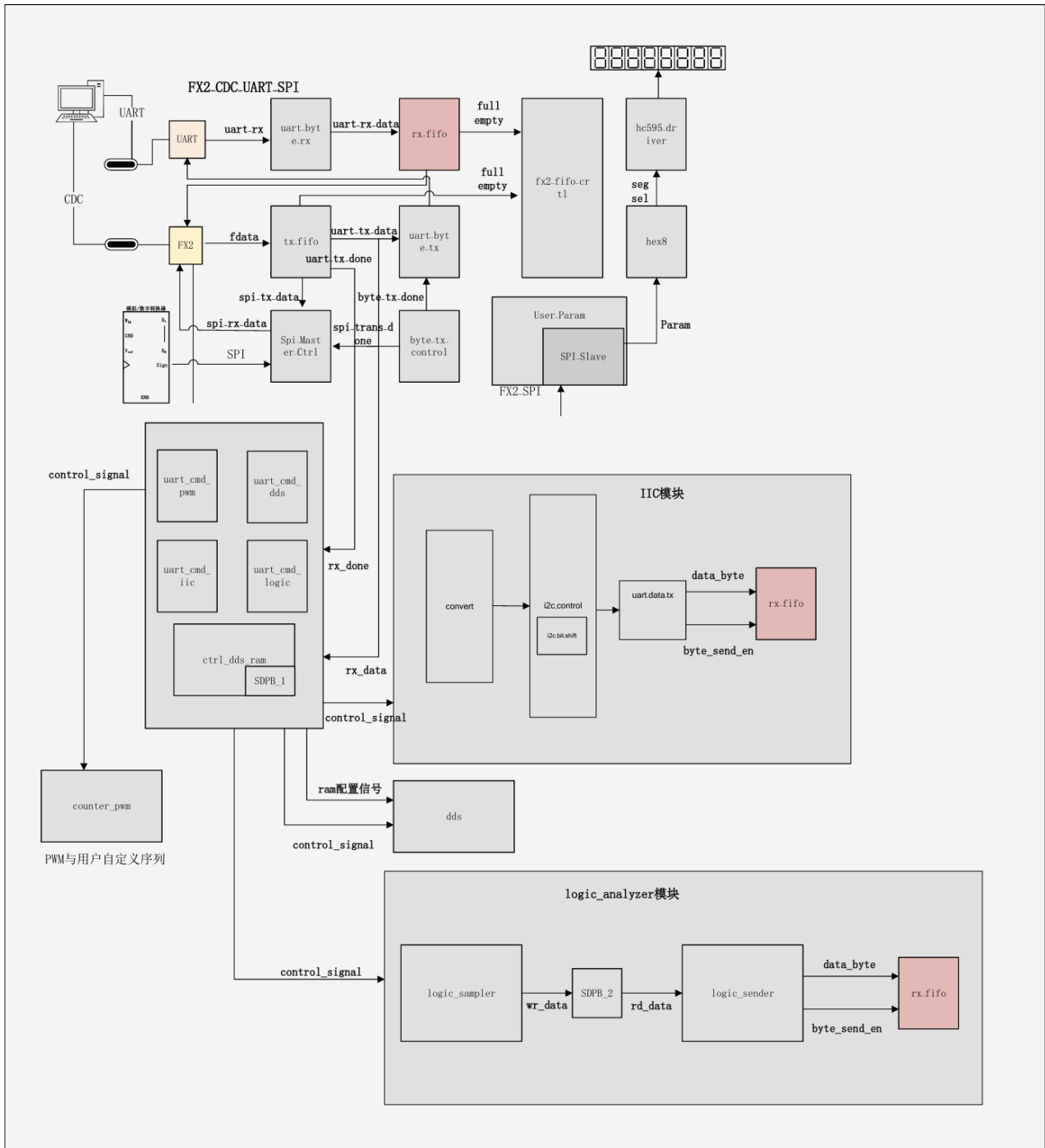


图表 2 硬件整体框图

示：

2.2.2 电路重要模块及其接口介绍：

(1)：USB 通信接口模块：USB 通信模块框图如图表 3 所示，其中的三个“rx_fifo”模块为同一模块，接入 USB 发送端发送数据至上位机。



图表 3 USB 通信控制模块

各个模块说明如下：

1. **uart_byte_rx** 模块：接收串口 UART 数据，发送至 **rx_fifo** 模块从而发送

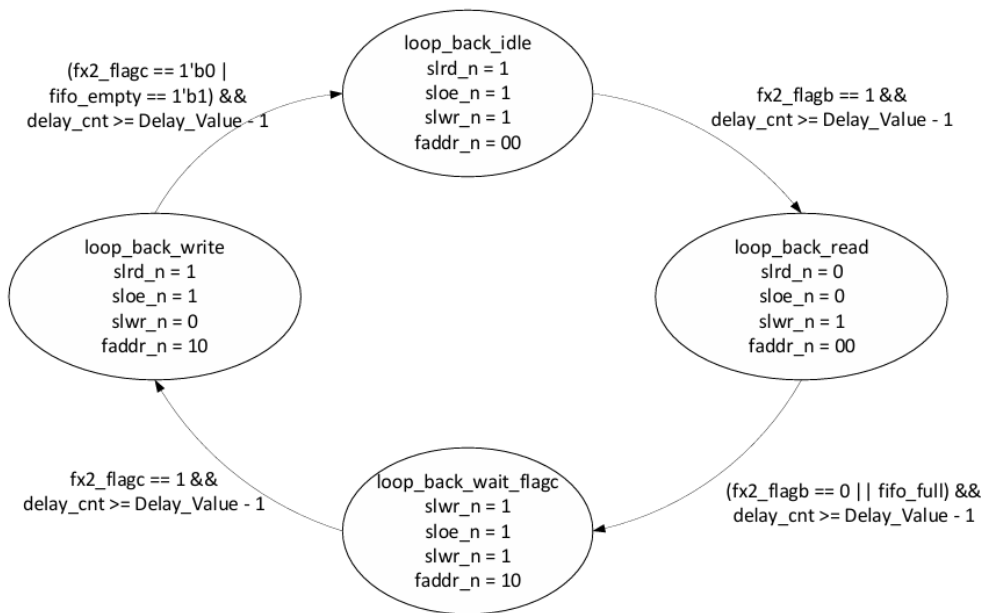
至 USB，实现 UART – USB。

2. `uart_byte_tx` 模块：从 `tx_fifo` 模块中接受数据，通过串口发送数据至电脑，实现 USB—UART

3. `rx_fifo` 模块：USB 的读 FIFO，USB 芯片从机从中读取数据并传输至上位机。

4. `tx_fifo` 模块：USB 的写 FIFO，USB 传输的字节数据通过 `tx_fifo` 传输至控制模块，从而实现 PWM，自定义序列，转 IIC，转 SPI，模拟波形输出等功能。

5. `fx2_fifo_ctrl`：FIFO 的控制模块，通过状态机来判断两个 FIFO 的空满状态进行控制读写，确保数据不会丢失。状态机状态转移图如图表 4 所示：



图表 4 状态机状态转移图

6. `User_Param` 模块：实现 SPI 从机，能够通过 SPI 接收来自 FX2 的串口参数并提供给外部模块，并能够接受外部模块提供的下位机串口参数，发送给

FX2 芯片。

7. hex8 和 hc595_driver 模块：数码管显示 User_Param 接收到的串口参数（波特率、停止位、校验位等）。

8. spi_Master_Ctrl 模块：实现 CDC 转 SPI，可设置 MOD 模式，可连续通信。

9. CMD 模块（uart_cmd_pwm、uart_cmd_dds、uart_cmd_iic、uart_cmd_logic、ctrl_dds_ram）：对 CDC 发送的串口命令进行协议解析从而控制后续模块，协议帧在 2.23 中进行详细介绍。

10. IIC 模块（包括 convert、i2c_control、uart_data_tx）：对解析的命令与对应的器件做 IIC 通信，写入数据或将数据读出并通过 rx_fifo 从 USB 发出。

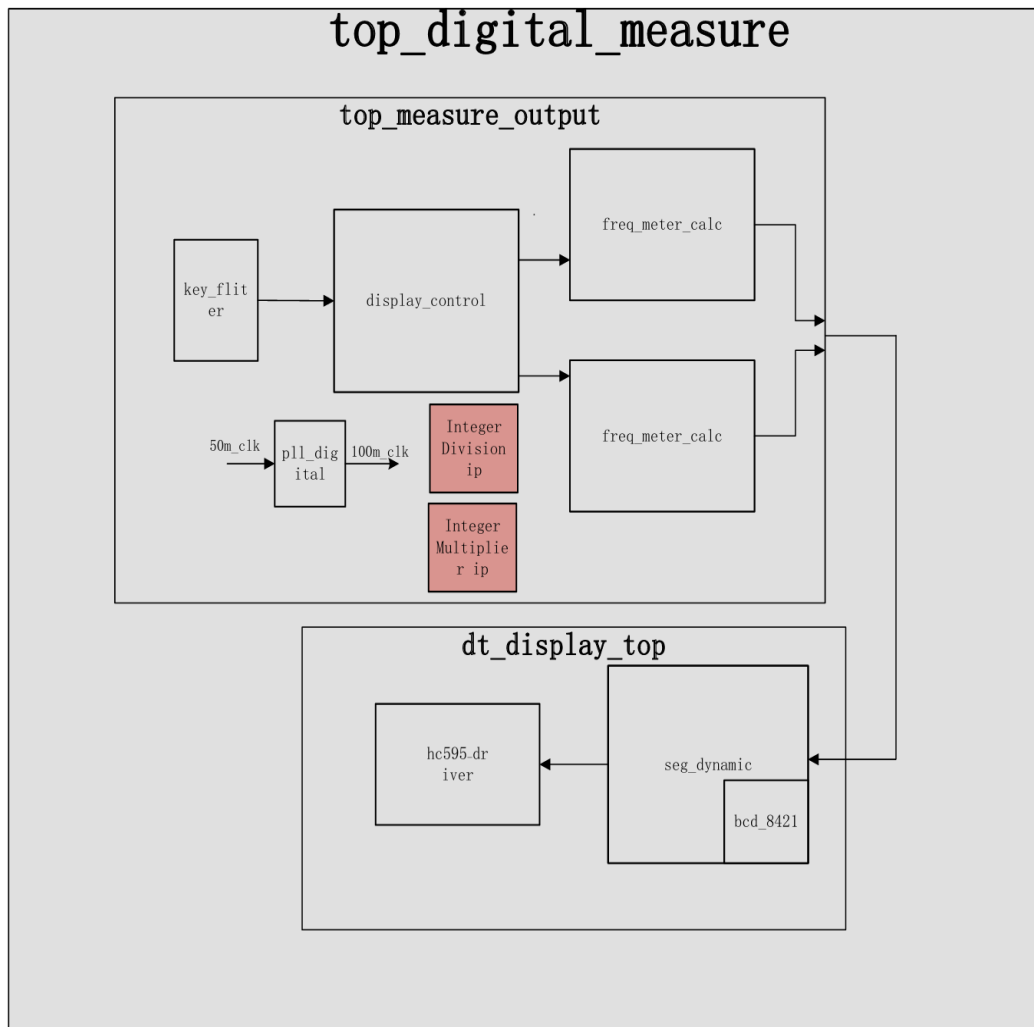
11. dds 模块：通过解析出的命令配置双通道输出的波形、频率、相位信息。并通过配置 SDPB1 伪双端口 ram 设置用户自定义波形

12. counter_pwm 模块：通过解析出的命令配置四路 PWM 和四路用户自定义序列输出。

13. logic_analyzer 模块：通过解析出的命令配置采样深度，触发模式，触发深度等信息，然后将采样数据先在 ram 中短暂存储，然后通过 rx_fifo 从 USB 发出。

(3): 数字信号测量模块框图如下图表 7 所示:

1. key_fliter 模块: 按键控制数码管当前显示什么 (对 key 信号进行消抖)。
2. display_control 模块: 通过 key 信号, 用状态机控制当前输入给 dt_display 的值。
3. freq_meter_calc 模块: 用 100MHz 的信号计数测量数字信号的高



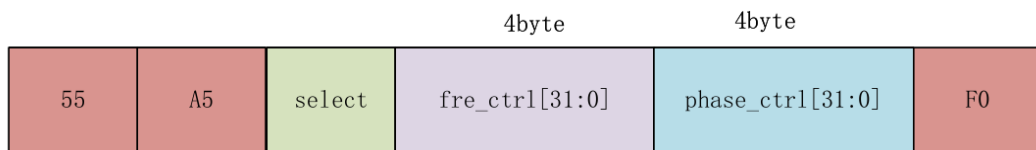
图表 7 数字信号测量模块框图

低电平时间，并通过整数除法 ip 核算出对应的占空比。

4. freq_meter_calc2 模块：通过等精度法测量数字信号的频率，理论可测频率无限大。

2.3 上下位机通信详细协议帧：

2.3.1 模拟信号发生器



图表 8 模拟信号发生器上下位机协议帧

如图所示，协议帧头固定为两字节十六进制数据 55 A5，select 字节的 0-4 位做波形选择，第 5 位做输出通道选择，第 6、7 位默认为 0。例如 0010_0001 为通道 2，第一个波形； 0000_0010 为通道 1，第二个波形。波形规定如下图所示：

```

parameter  sin_wave    = 5'b00001    , //正弦波
            squ_wave    = 5'b00010    , //方波
            tri_wave    = 5'b00100    , //三角波
            saw_wave    = 5'b01000    , //锯齿波
            half_sin_wave = 5'b10000    ; //半正弦波
    
```

图表 9 波形选择定义

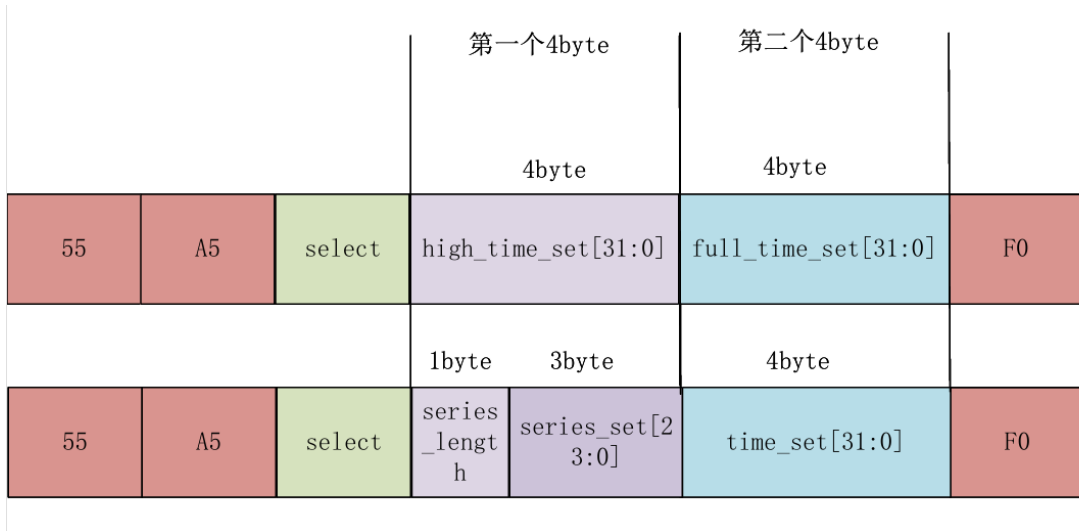
fre_ctrl 长度为 4 个字节，用来控制输出波形的频率；phase_ctrl 长度也为 4 个字节，用来控制输出波形的相位。二者的计算可以由以下公式得到：

$$fre_ctrl = \frac{freq_out \times 2^{32}}{50 \times 10^6}$$

$$phase_ctrl = \frac{phase \times 2^{32}}{360}$$

其中，freq_out 和 phase 是用户设定的频率和相位。最后的帧尾固定为 F0。于是每次上位机向下位机发送一次 12 字节长的命令。

2.3.2 PWM 和自定义序列发生器



图表 9 PWM（上）与自定义序列（下）发送上下位机协议帧

如图所示，协议帧头固定为两字节十六进制数据 55 A5，select 字节为选择信号，选择的逻辑如下所示：

```

select = 8' b0000_0001 //选择第一路PWM输出
select = 8' b0000_0010 //选择第二路PWM输出
select = 8' b0000_0100 //选择第三路PWM输出
select = 8' b0000_1000 //选择第四路PWM输出
select = 8' b0001_0000 //选择第一路用户自定义序列输出
select = 8' b0010_0000 //选择第二路用户自定义序列输出
select = 8' b0100_0000 //选择第三路用户自定义序列输出
select = 8' b1000_0000 //选择第四路用户自定义序列输出
    
```

根据 select 的信号值不同，如果此时是控制 PWM 输出，则第一个 4byte 用于确定高电平时间。第二个 4byte 用于确定一个周期时间（高低电平总时

间)；如果此时是控制用户自定义序列输出，则第一个 4byte 高 1byte 位数据输入序列长度，低 3byte 用于存储用户自定义序列。第二个 4byte 用于控制一次信号所占时间。无论是哪个模式，最后的帧尾都是 F0。

例如：select = 8'b0001_0000, 则此时控制用户自定义序列输出，假设用户要输入序列为：0000_0001_0000_0010_0001，则用户输入序列长度 (series_length) 为 20 位，则第二个 byte 为 14 00 10 21，则循环输出 0000_0001_0000_0010_0001.....，且每次“1”和“0”的时间与 time_set[31:0] 有关。

time_set, high_time_set, full_time_set 满足以下计算公式：

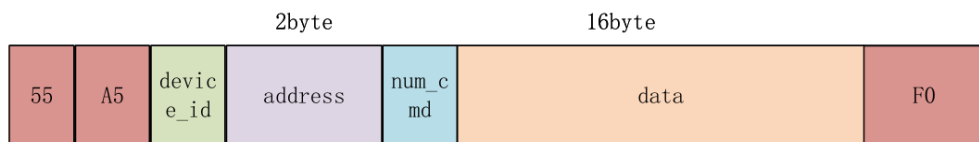
$$\text{time_set} = \text{用户设置时间} \times 50 \times 10^6$$

$$\text{high_time_set} = \text{用户设置高电平时间} \times 50 \times 10^6$$

$$\text{full_time_set} = \text{用户设置一个周期时间} \times 50 \times 10^6$$

用户输入的时间单位规定为 ms，精度为 0.001ms，即 1μs，在上位机程序中转化为 s 发送给下位机。

2.3.3 CDC 转 IIC 读写控制器



图表 10 CDC 转 IIC 上下位机协议帧

如图所示，协议帧头固定为两字节十六进制数据 55 A5，device_id 表示器件地址，由用户在用户端输入。address 为 2 字节长度，操作对应的该器

件中寄存机/memory 地址 (16bit)。num_cmd 为长 1 字节的功能码，最高位控制读写 (0 代表写，1 代表读)，低七位代表读写数据个数。其后紧接着 16 个字节长度的由用户输入的数据，如果用户输入的数据不足 16byte，则需要补齐命令，即数据部分长度必须为 16byte，不足的部分补充 0。

2.3.4 CDC 转 SPI 控制器



Bit7(MSB)	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
DONTC	DONTC	ADD2	ADD1	ADD0	DONTC	DONTC	DONTC

图表 11 CDC 转 SPI 上下位机协议帧

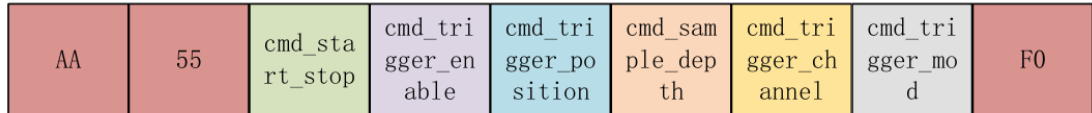
协议没有固定的帧头，帧尾固定为 00，第一个字节为 channel_set,其第 3 位到第 5 位用来确定用户选择的通道数，其余位都固定为 0。用户界面设置通道 (0-7) 以及采样几次，采样几次就循环发送几次上面的协议帧内容，一次发送的字节长度为 2。

2.3.5 手绘波形发送器

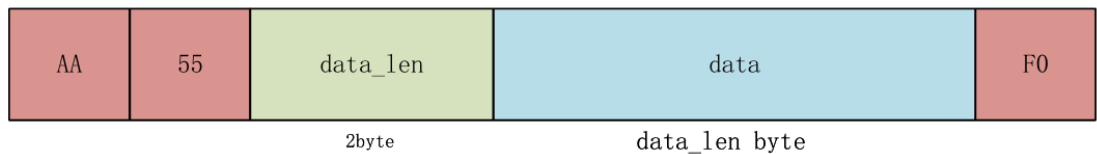
协议没有固定的帧头帧尾，当用户在绘图区手绘出波形时，上位机会自动对曲线等间距进行 512 个点的采样，并把绘图区在高度上进行 256 个等分，对于每个采样点，取值为距离它最近的量化点，以此来实现对手绘波形的数字量化。根据量化点还原出用户的曲线后，还会进过一系列高斯平滑滤波来消除曲线上的噪点，使还原出的波形尽可能平滑。最后在两个

相邻的量化点之间，会均匀地插入 8 个点，实现 8 倍插值，最后当用户按下“发送”按键时，将会向上位机一次性发送 4096 个数据点。

2.3.6 逻辑分析仪



图表 12 上位机向 FPGA 发送控制命令帧



图表 13 FPGA 向上位机发送数据命令帧

如图 12 所示，上位机向 FPGA 发送控制命令帧结构由 9 个字节构成，每个字节均有特定含义：帧起始由固定的 0xAA（字节 0）和 0x55（字节 1）标识，命令类型为 0x01（字节 2）以表示开始采样，若为 0x02 则表示停止采样；触发使能（字节 3）可取 0x01（启用触发采集）或 0x00（禁用触发，即自由采集）；触发位置（字节 4）取值范围为 0x00 至 0x04，分别对应 0%、25%、50%、75%和 100%的窗口位置；采样深度（字节 5）取值 0x00 至 0x04 分别代表 1000 点、5000 点、10000 点、20000 点和 50000 点；触发通道（字节 6）限定在 0x00 至 0x07，对应通道 0 至 7；触发方式（字节 7）取值范围 0x00 至 0x04，分别代表上升沿、下降沿、高电平、低电平与双边沿触发；最后，帧尾以固定值 0xF0（字节 8）结束整个数据帧。

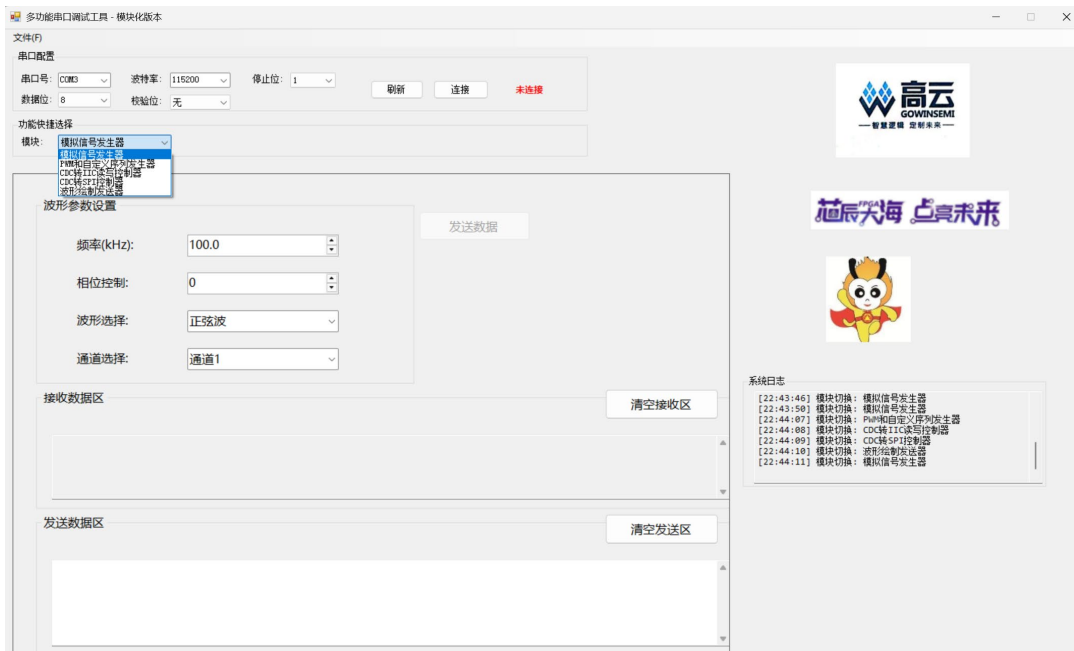
如图 13 所示，FPGA 向上位机发送的命令帧以固定的起始标识 0xAA（字节 0）和 0x55（字节 1）开头；紧随其后的是两个字节共同表示的数据长度，

其中字节 2 为数据长度的高 8 位（Len_H），字节 3 为数据长度的低 8 位（Len_L），二者采用大端序（Big-Endian）共同组成一个 16 位的长度值，用以指示后续采样数据部分的总字节数；从字节 4 开始直至帧尾则为变长的实际采样数据区，其具体长度由之前计算出的 Len_L 的值确定，每个数据字节的取值范围在 0x00 到 0xFF 之间。

2.4 软件系统介绍

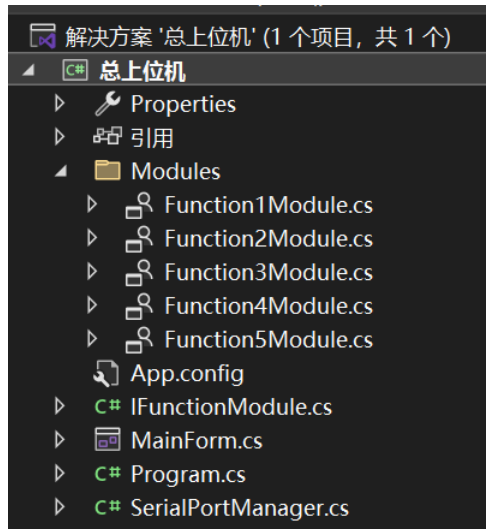
2.4.1 软件整体介绍（含 PC 端或云端，结合关键图片）；

上位机有 5 个输出模式，分别是模拟信号发生器、PWM 和自定义序列发生器、CDC 转 IIC 读写控制器、CDC 转 SPI 控制器、波形绘制发送器，在主界面可以由用户对这 5 个模式进行选择。用户可以在主界面选择串口号、波特率等参数，切换功能时，不需要断开/重连串口，如图所示。

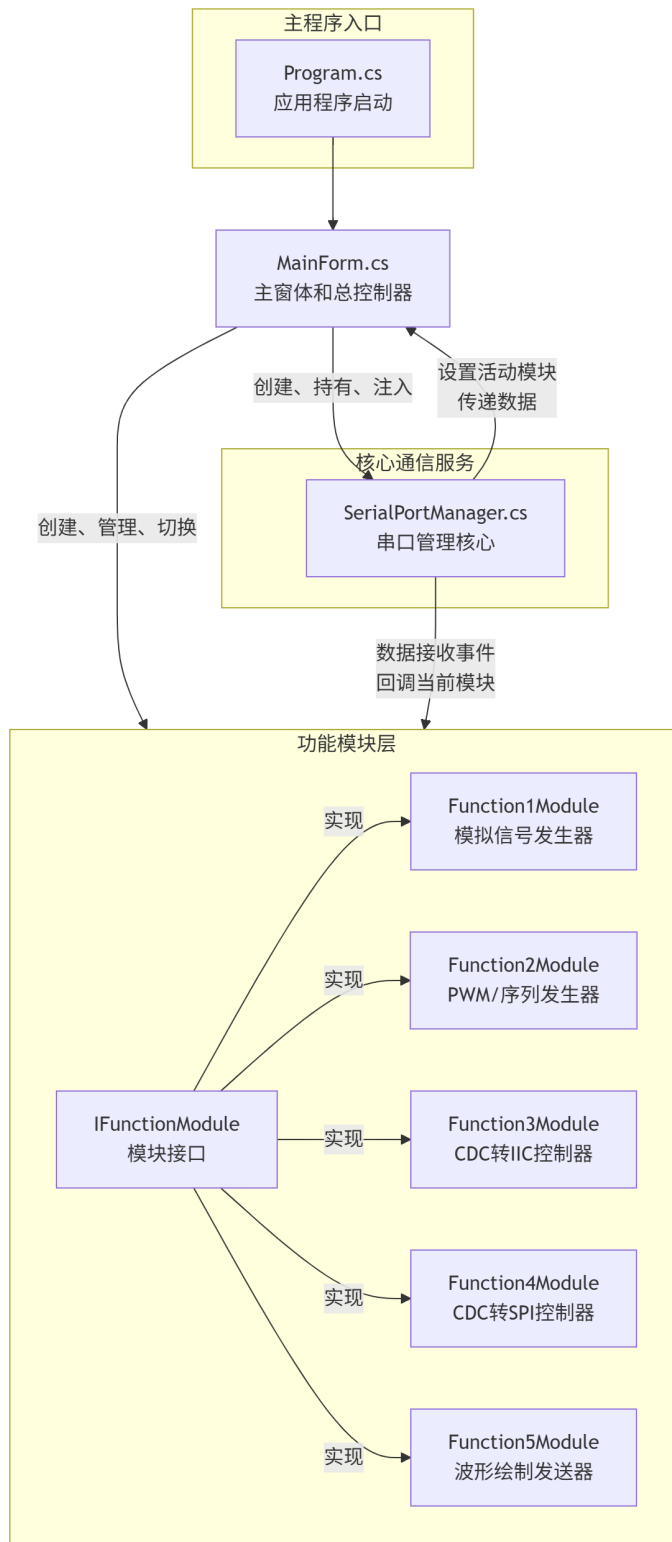


图表 14 上位机整体界面

上位机主要由主程序入口 Program.cs、主窗体控制器 Mainform.cs、串口管理核心 SerialPortManager.cs、模块接口 IfunctionMoudule.cs 以及 5 个实现具体功能的 FunctionModule 模块组成，如图 15 所示



图表 15 代码架构



图表 16 上位机设计总体思路框图

图中实线箭头表示控制流/依赖关系，Program 启动 MainForm 主程序。MainForm 是核心控制器，它创建并持有 SerialPortManager 和所有 IFunctionModule 实现实例。虚线箭头则代表了数据流，SerialPortManager 从物理串口接收数据，然后通过调用当前活动模块的 OnSerialDataReceived 方法将数据传递给具体的功能模块处理。模块也可以通过 SerialPortManager 的 SendData 方法发送数据。

2.4.2 软件各模块介绍

2.4.2.1 核心接口与主控模块 (IFunctionModule, MainForm)

作为整个架构的基石，IFunctionModule 接口定义了所有功能模块必须遵守的契约，实现了“面向接口编程”，使得主窗体与具体功能解耦。MainForm 作为主控制器，不关心各模块的具体功能，只负责模块的加载、界面的切换以及串口管理器的生命周期管理，严格遵循“单一职责原则”。MainForm 通过 InitializeModules 方法创建所有模块实例，并通过 SetSerialPortManager 方法将唯一的串口管理器实例“注入”到每个模块中（依赖注入），确保了通信资源的一致性。

2.4.2.2 串口通信服务模块 (SerialPortManager)

该类封装了所有底层串口操作，采用单例模式（由主窗体确保唯一性）管理串口连接、数据收发。它使用事件驱动的 DataReceived 函数，高效且稳定。其核心职责是作为可靠的通信通道，并不处理任何业务逻辑。其中的 SetActiveModule 方法允许主窗体动态设置当前接收数据的模块，实现了数据流的动态路由。

2.4.2.3 五个具体功能模块 (Function1Module~ Function5Module)

这五个模块是软件功能的最终承载者。每个模块都继承自 `UserControl` 并实现 `IFunctionModule` 接口，因此每个模块都是兼具独立用户界面和业务逻辑的完整功能单元。它们通过共享的 `SerialPortManager` 实例进行通信，但彼此功能完全独立，实现了“高内聚、低耦合”。

Function1Module (模拟信号发生器):根据 UI 参数生成面向 FPGA 的固定格式波形控制协议。

Function2Module (PWM 和自定义序列发生器):支持 PWM（占空比/高电平时间两种模式）和自定义序列两种信号生成模式，协议设计巧妙。

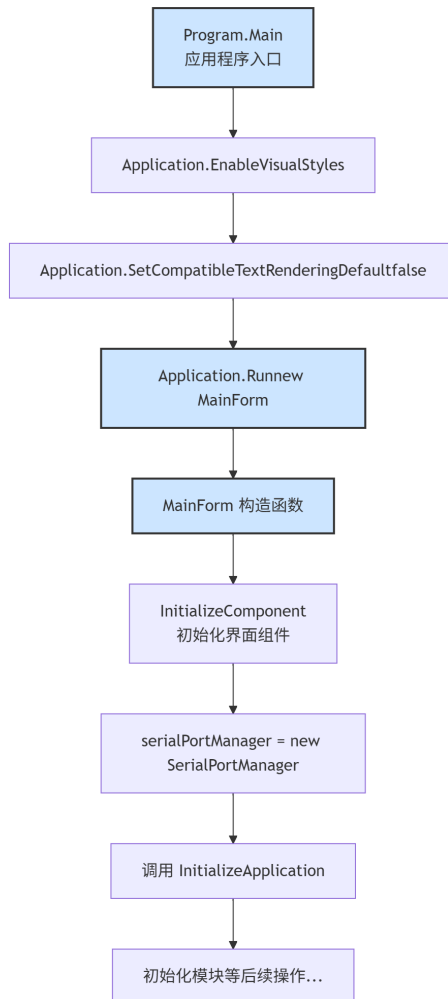
Function3Module (CDC 转 IIC 读写控制器):模拟 USB CDC 到 I2C 的转换，实现 I2C 设备的读写操作，协议固定为 23 字节，具有严格的输入验证。

Function4Module (CDC 转 SPI 控制器):实现简单的 SPI 通道选择命令发送，支持批量发送，协议简洁。

Function5Module (波形绘制发送器):最具创新性，将图形绘制转化为波形数据，并集成多种专业数字信号处理算法（滤波、插值）。

2.4.3 流程分析

2.4.3.1 顶层流程：应用程序启动与初始化



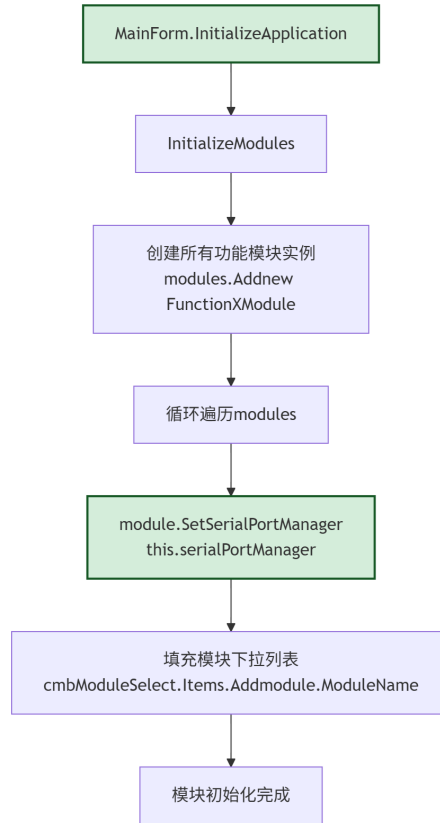
图表 17 程序初始化流程

完成 Windows Forms 应用程序的启动环境配置，并创建应用程序的主窗口。

程序的执行始于 **Program.Main** 函数，该函数作为应用程序入口，主要任务是配置 Windows 窗体应用程序的视觉样式并启动消息循环，其输入输出参数均为空，流程结束时将主窗体呈现给用户。主窗体的初始化由 **InitializeApplication** 函数完成，该函数内部调用了 **InitializeModules** 函数，其流程是循环创建所有功能模块实例，并将串口管理器实例注入到每个模块中，最后填充模块选择下拉列表，此过程无需显式输入，输出结果是初始化就绪的模块集合和界面控件。

2.4.3.2 核心控制流：主窗体初始化与模块加载

动态创建所有功能模块，建立模块与串口管理器的关联，并在主界面上初始化模块选择列表。



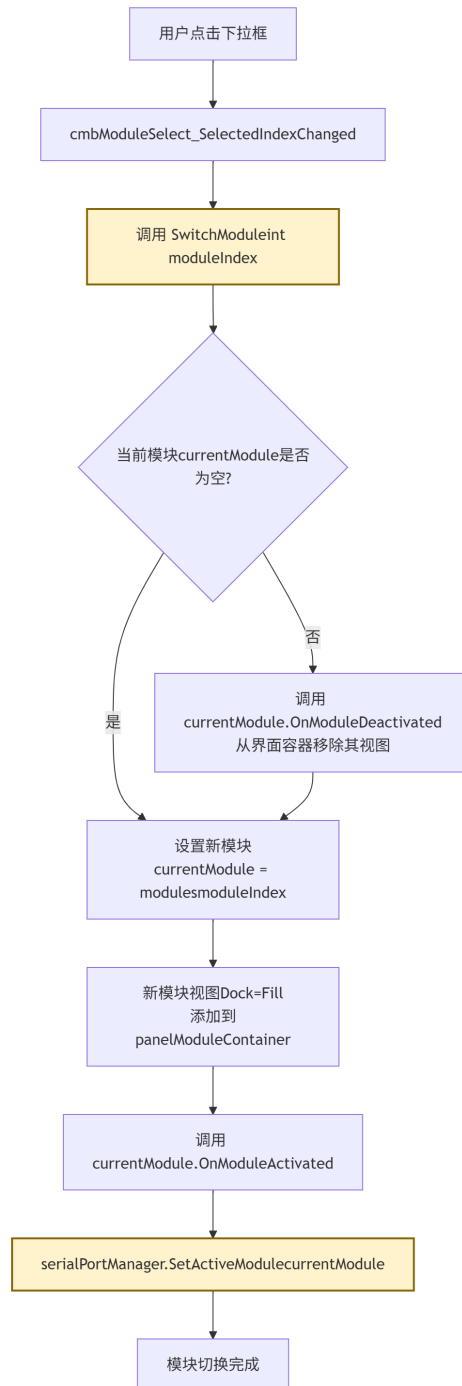
图表 18 主窗体初始化与模块加载流程

首先通过 `new` 关键字依次创建 `Function1Module` 到 `Function5Module` 的实例，并添加到 `List<IFunctionModule> modules` 集合中。此集合是主窗体管理所有功能模块的容器。然后遍历 `modules` 集合，并同时为每个模块调用其 `SetSerialPortManager(this.serialPortManager)` 方法。这是依赖注入的关键一步，它将唯一的 `SerialPortManager` 实例传递给每个模块，确保所有模块共享同一个通信通道，避免了资源冲突。然后再次遍历 `modules` 集合，将每个模块的 `ModuleName` 属性（如“模拟信号发生器”）添加到主界面的模块选择下拉框 (`cmbModuleSelect.Items`) 中。这为用户提供了可视化的模块切换入口。最后设置默认模块，将下拉框的选中索引 (`SelectedIndex`) 设置为 0，即默认选中并激活。

活第一个模块。

2.4.3.3 核心控制流：模块切换

响应用户操作，安全地卸载当前活动模块的界面和状态，并加载、激活新



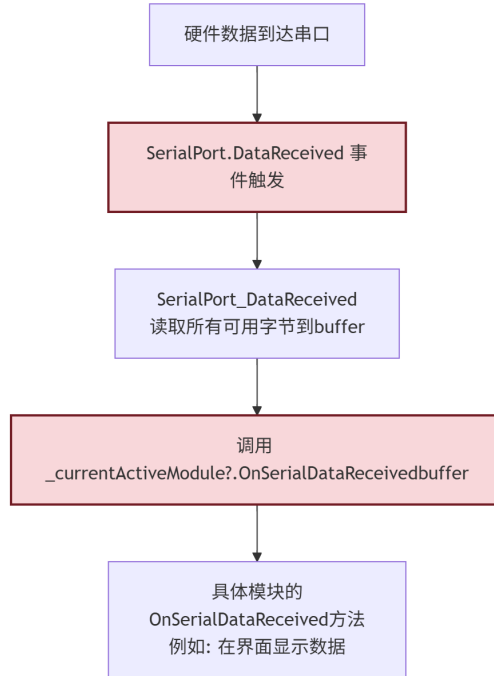
图表 19 模块切换流程

选中的模块。

这是体现模块化架构精髓的核心流程。首先判断当前是否有活动模块 (if (currentModule != null))。如果有，则调用当前活动模块的 OnModuleDeactivated() 方法。这是给模块一个信号，让其执行必要的清理工作，例如停止定时器、重置状态等。然后从主窗体的界面容器 (panelModuleContainer.Controls) 中移除当前模块的用户控件 (currentModule.ModuleView)。将旧模块的界面从屏幕上清除。然后开始启用新模块，根据传入的 moduleIndex，从 modules 列表中获取对应的模块实例，并将其赋值给 currentModule 变量。将新模块的用户控件 (currentModule.ModuleView) 添加到主窗体的界面容器中，并设置其停靠属性为 DockStyle.Fill，使其充满整个内容区域。调用新模块的 OnModuleActivated() 方法。这是给模块一个信号，让其执行初始化工作，例如根据当前串口连接状态更新按钮可用性。调用 serialPortManager.SetActiveModule(currentModule)。这是至关重要的一步，它告知串口管理器，此后接收到的所有串口数据都应传递给这个新激活的模块处理，从而实现了数据流的动态切换。

2.4.3.4 核心数据流：串口数据接收与分发

当串口接收到数据时，由串口管理器将数据可靠、及时地传递给当前正在前台活动的功能模块进行处理。

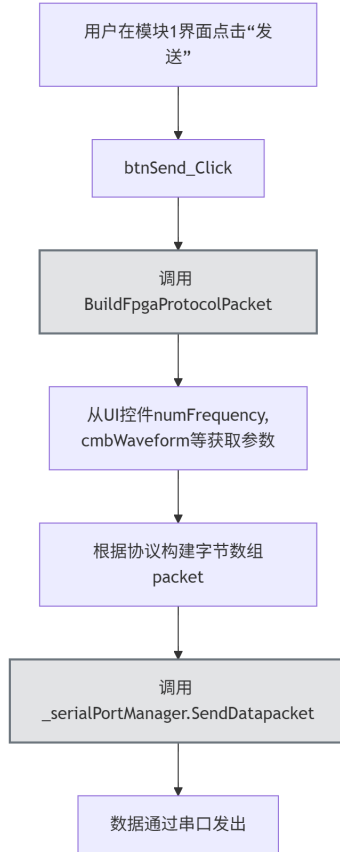


图表 20 串口数据分发流程

串口数据接收流程由串口管理器的 `SerialPort_DataReceived` 事件处理函数驱动。当硬件数据到达时，系统在辅助线程上触发此事件，该函数读取串口缓冲区中的所有可用字节到一个字节数组中，然后立即调用当前活动模块的 `OnSerialDataReceived` 方法，将原始数据字节数组作为参数传递给它。模块自身的 `OnSerialDataReceived` 方法首先进行线程安全检查，通过 `Invoke` 方法确保在 UI 线程上执行实际的数据处理逻辑，例如将字节数据解析并显示在文本框中。

2.4.3.4 底层业务流：功能模块发送数据（以 Function1 为例）。

根据用户在模块界面上的设置，构建特定的二进制命令帧，并通过串口管理器发送出去。

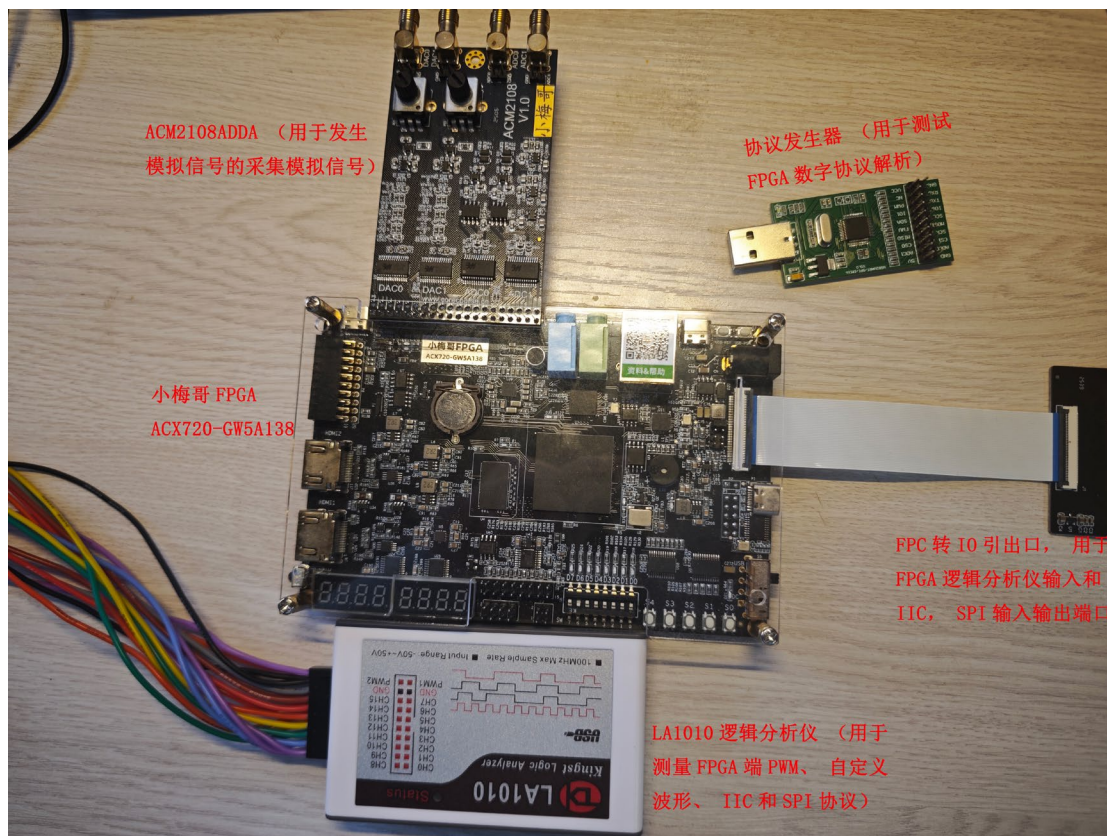


图表 21 功能模块发送数据流程（以 Function1 为例）

以 Function1Module 的按钮点击事件为例，始于用户交互。btnSend_Click 函数被触发后，首先调用 BuildFpgaProtocolPacket 函数，该函数从用户界面控件获取各种参数，按照预定义的 FPGA 协议格式构造一个完整的字节数组作为输出。随后，该字节数组被传递给串口管理器的 SendData 函数。SendData 函数接收此字节数组作为输入，在内部进行串口连接状态检查后，调用底层串口写入方法将数据发送出去，若发送失败则抛出异常。

3 第三部分 完成情况及性能参数

3.1 整体介绍（整个系统实物的正面、斜 45° 全局性照片）



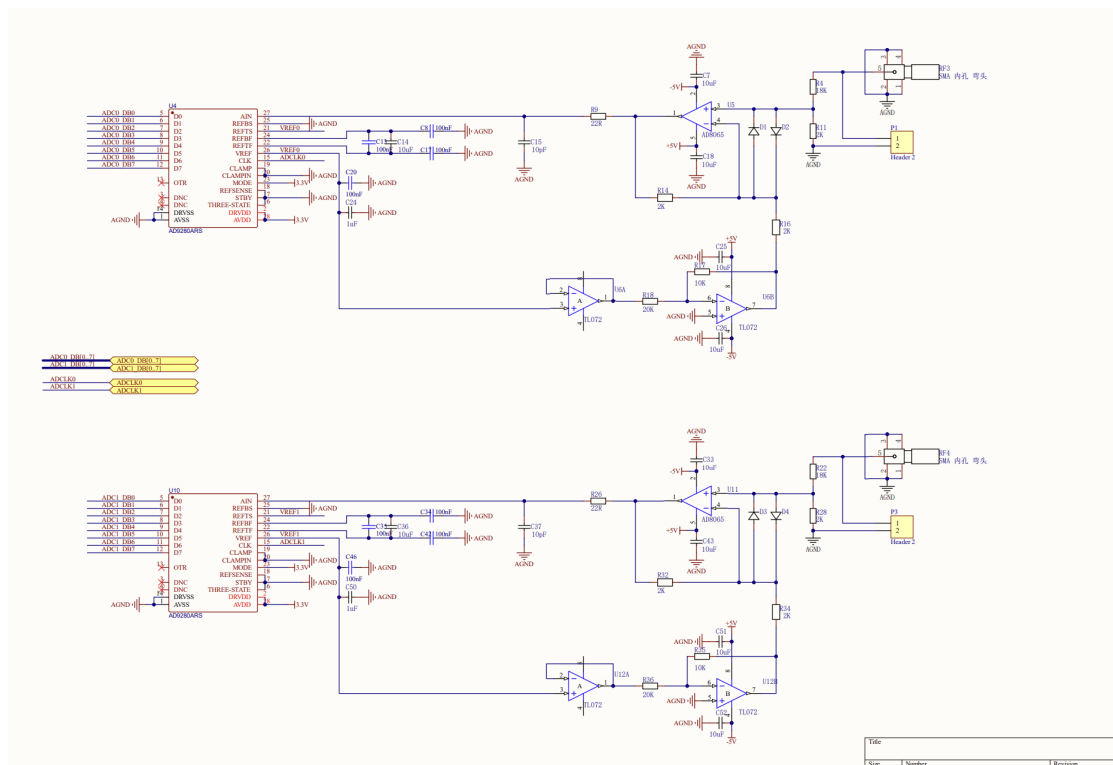
图表 22 作品整体组成部分

作品整体组成如图表 22 所示。

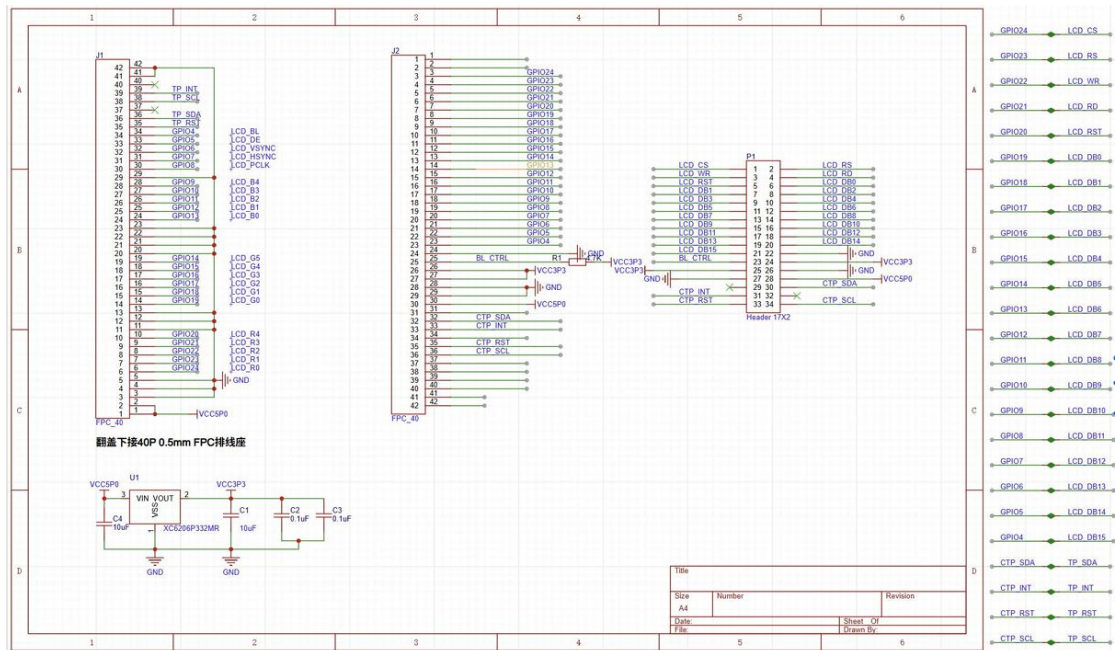
3.2 工程成果（分硬件实物、软件界面等设计结果）

3.2.1 电路成果；

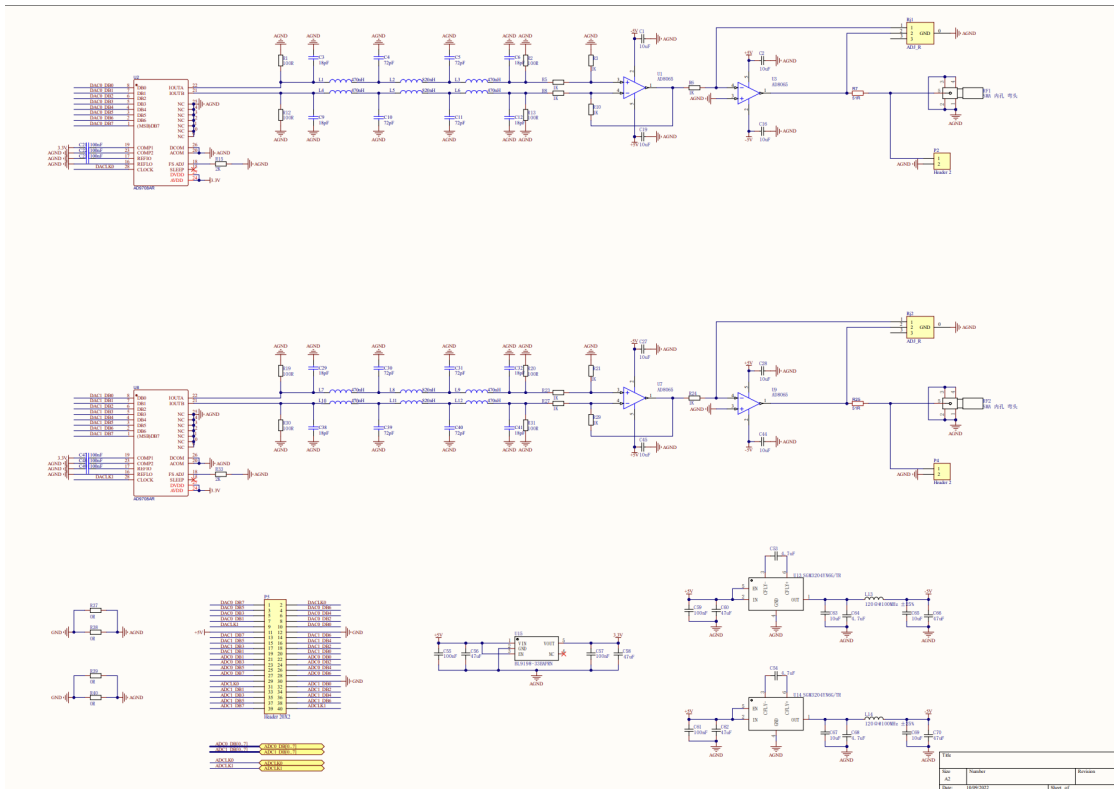
本设计主要为 FPGA 内部 RTL 级别设计，只有外挂 ACM2108 及 FPC 转 IO 口模块，这两个模块的电路原理图如图 23 24 25 所示：



图表 23 ACM2108 AD 部分原理图



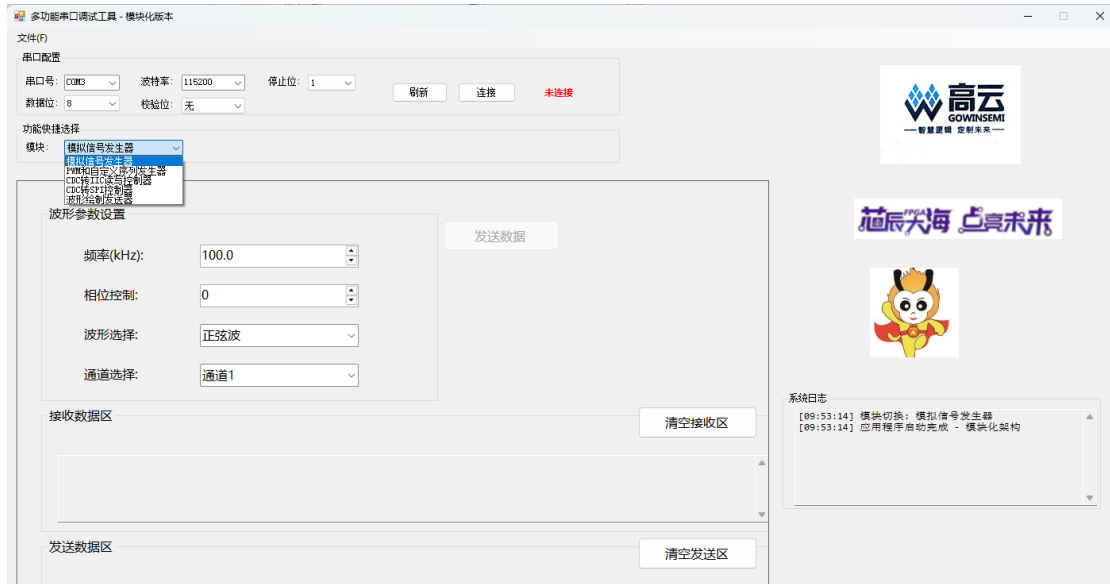
图表 24 FPC 转 IO 原理图



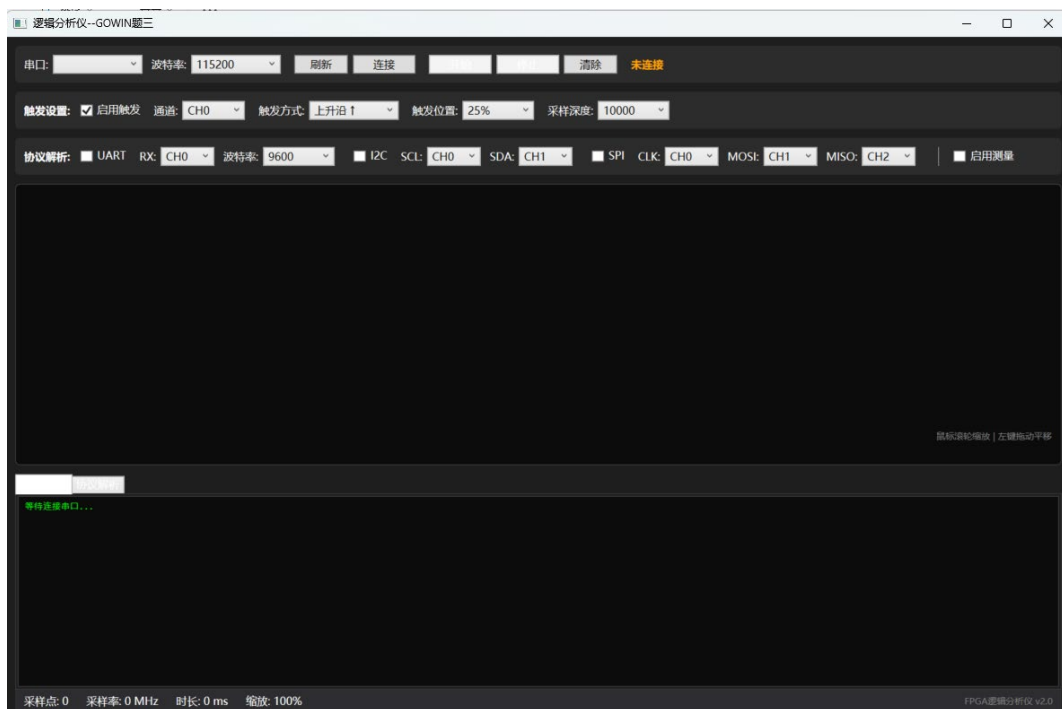
图表 25 ACM2108 DA 部分原理图

3.2.2 软件成果：

软件成果下图所示：



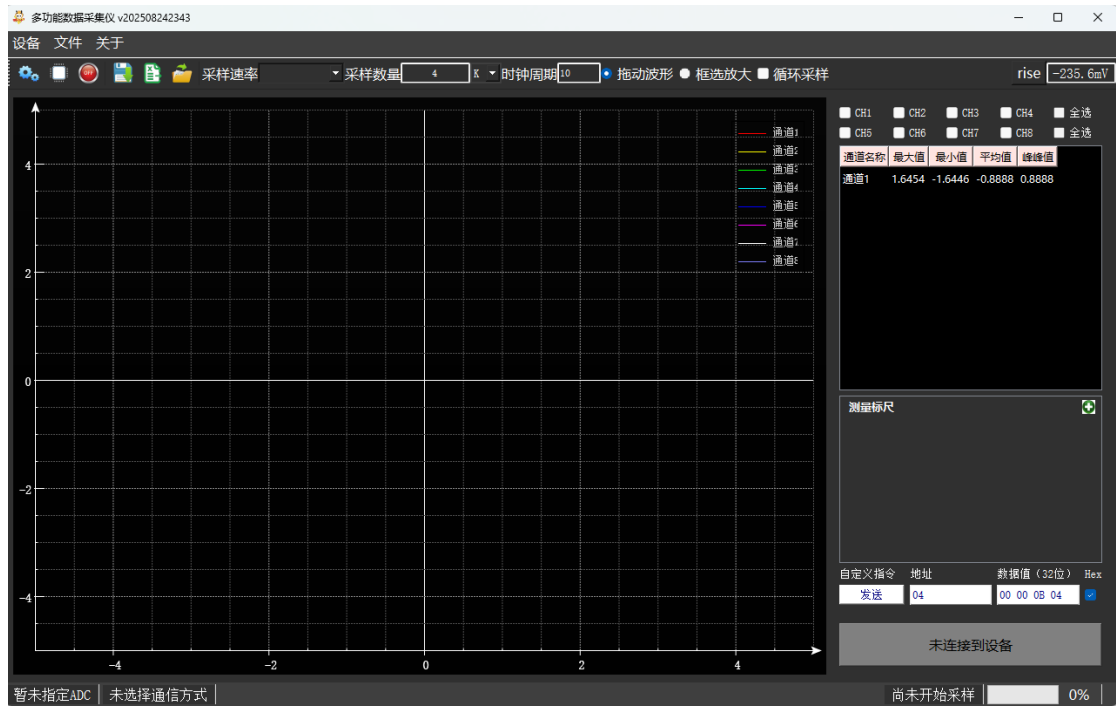
图表 26 CDC USB 部分上位机



图表 27 逻辑分析仪部分上位机

CDC USB 部分上位机如图表 26 所示：

逻辑分析仪部分上位机如图表 27 所示：

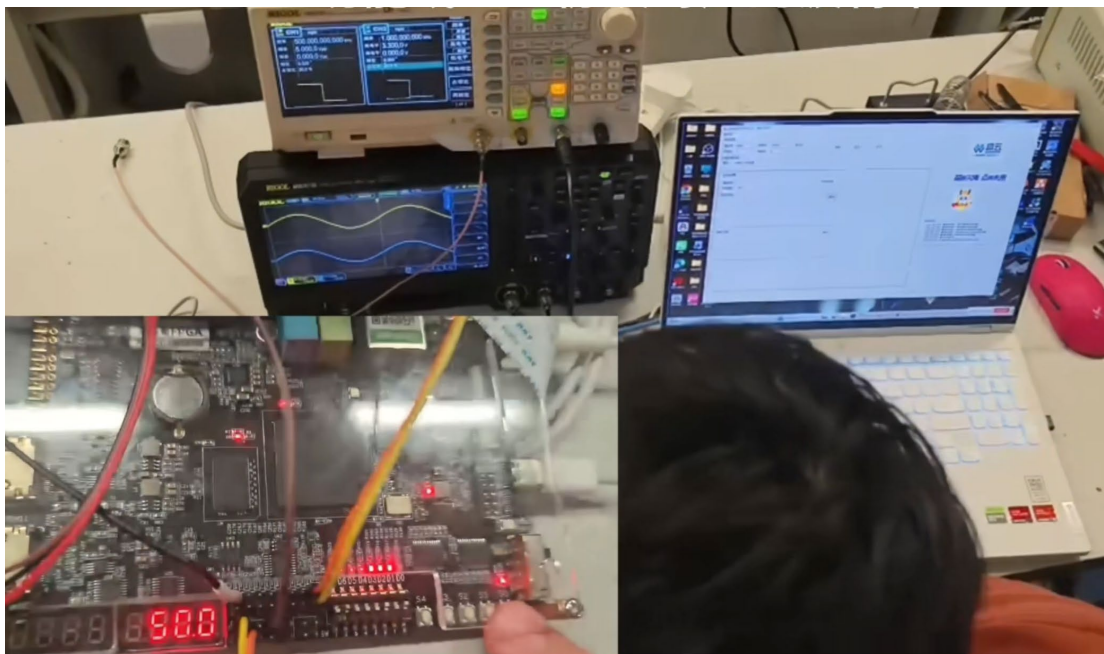


图表 28 以太网传输 ADC 部分上位机

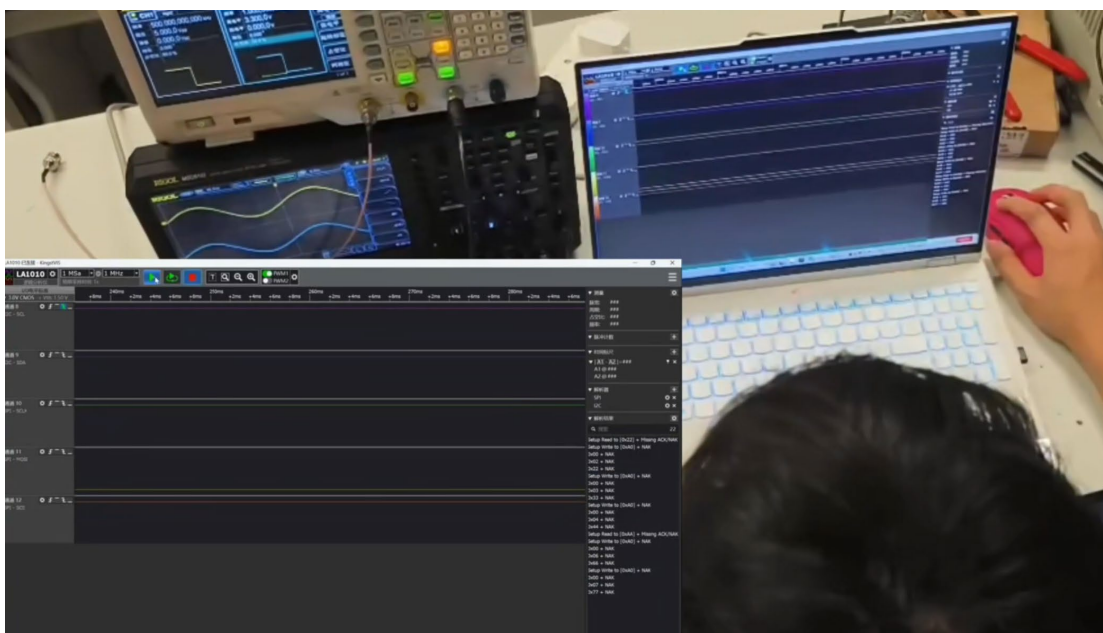
以太网传输 ADC 部分如图表 28 所示。

3.3 特性成果（逐个展示功能、性能参数等量化指标）

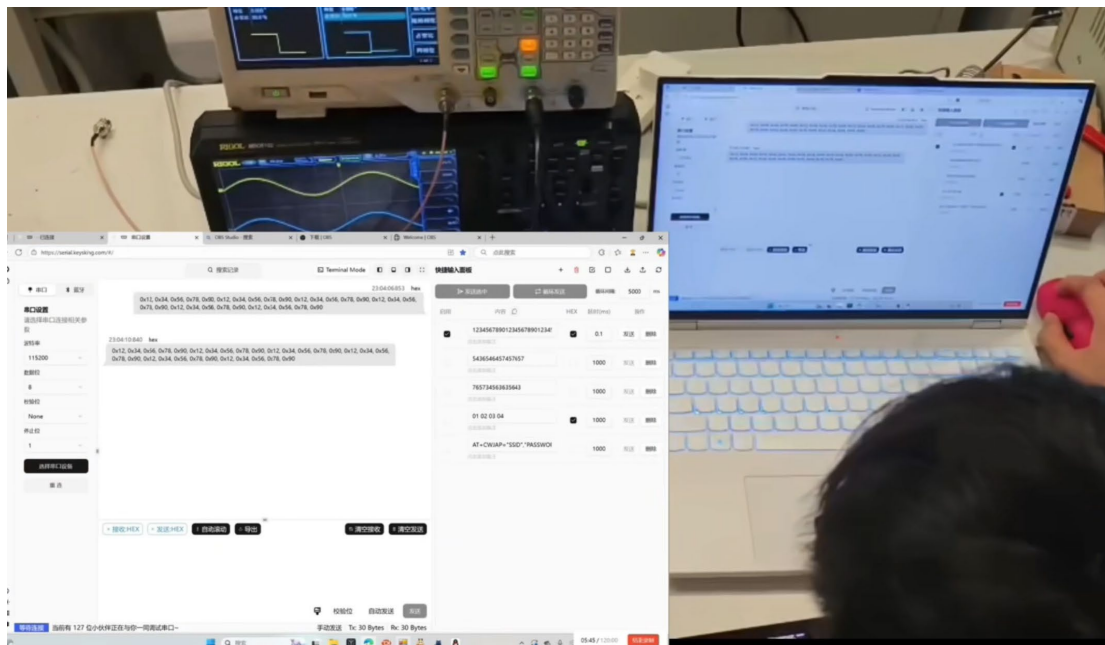
参数性能等量化指标已在第一部分中列表展示。功能展示如图：



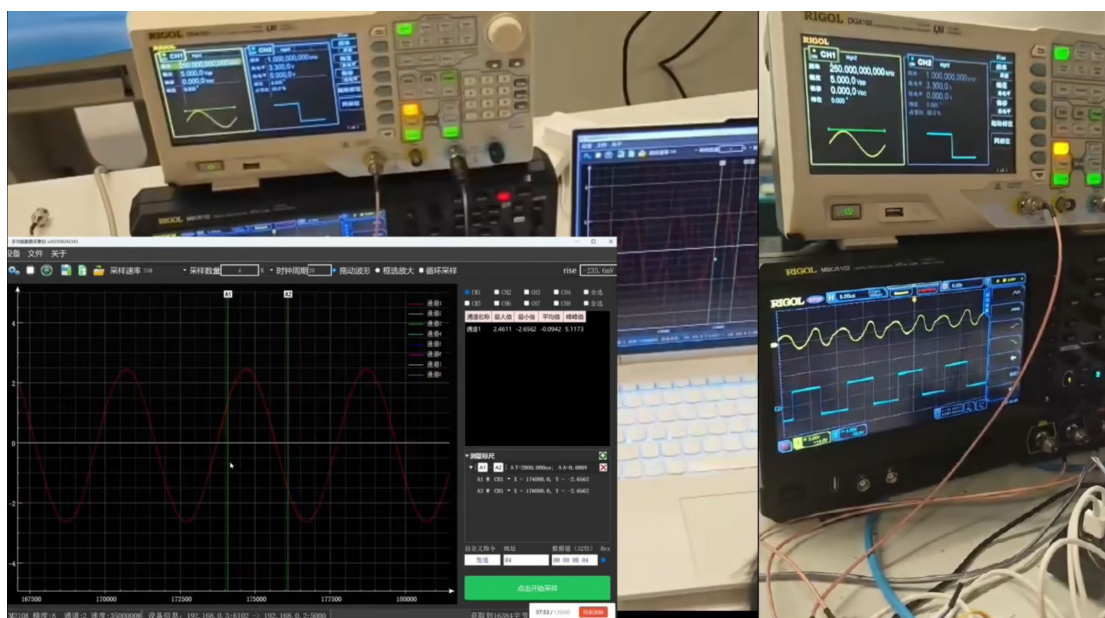
图表 29 数字信号测量



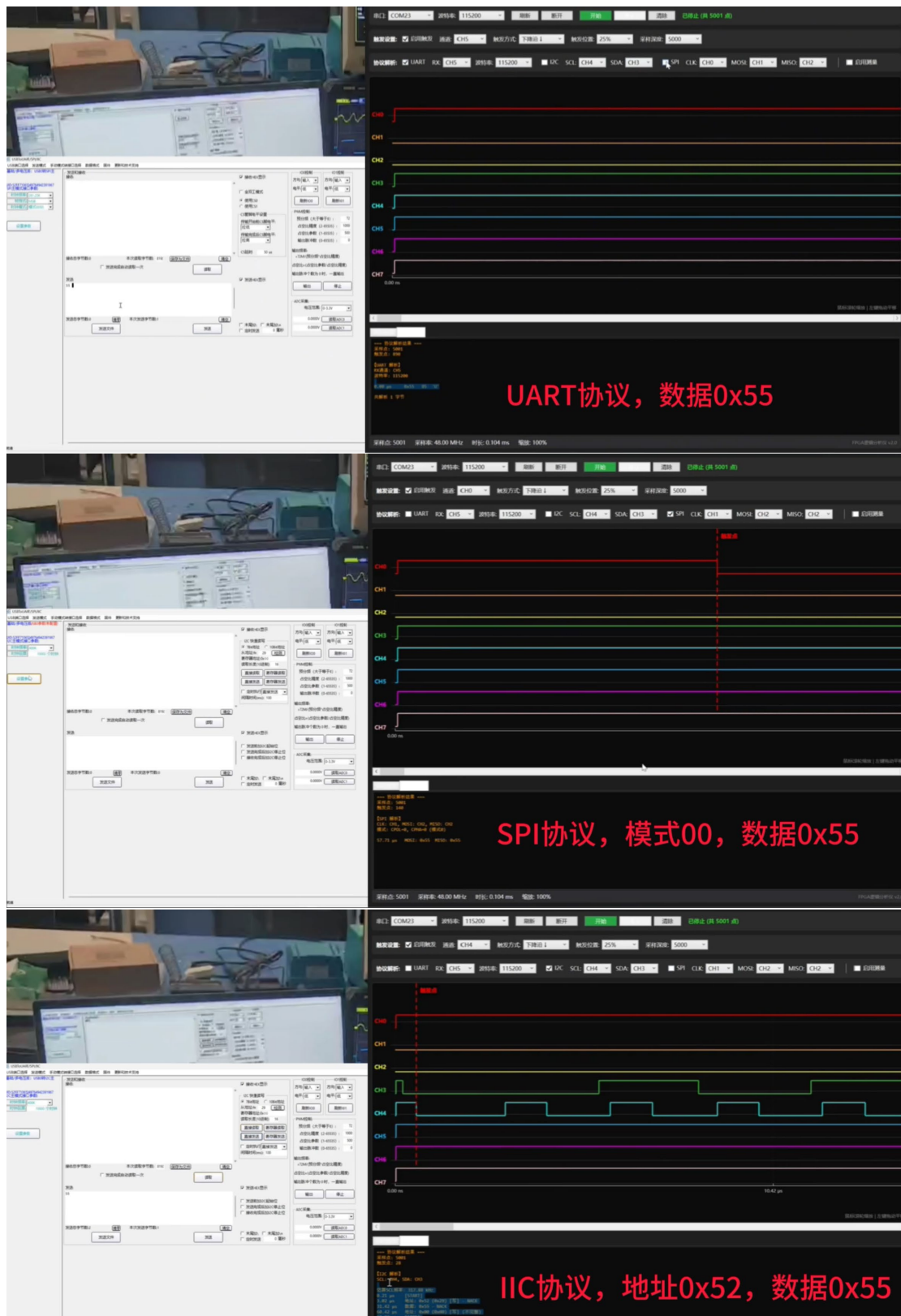
图表 30 CDC 转协议部分



图表 31 CDC 转串口



图表 32 ADC 和 DAC



图表 33 逻辑分析仪

4 第四部分 总结

4.1 可扩展之处

我们的系统现在仍需要通过滑动 SW 来控制选择 CDC 转 SPI、IIC、UART、PWM 等功能，后期我们可以继续优化系统，通过状态机控制，在接收字节命令时新加一个状态控制字节，从而提高使用便捷性，降低用户使用难度。

目前我们还未实现 ADC 采样的频谱显示，后期预备优化上位机的频谱显示；同时对于逻辑分析仪的上位机，我们目前在接收过多数据时会比较卡顿，后续预备更换框架，解决卡顿问题。

同时，我们的系统只有 SPI、IIC 主部分，没有从部分，或许后期可以增加从部分。

4.2 心得体会

在上位机开发初期，由于初次接触 C#语言，对语法和开发环境不熟悉，调试过程中常遇到难以理解的报错，代码屡次无法正常运行，一度带来较大挫败感。但正是这些困难锻炼了我独立解决问题的能力。我学会了高效搜索技术资料、查阅官方文档，并在开发者社区中寻求帮助，通过不断尝试和总结，提升了分析问题和定位错误的能力。

在整合上位机功能的过程中，我深刻体会到工程规范化的重要性。尤其是在初次编写代码时，如果没有统一的函数命名规则、清晰的文件结构和模块化的整理方法，后续的维护和功能扩展会变得异常困难。因此，我开始有意识地注重代码风格的一致性，并逐步建立规范的工程目录，这使得代码的可读性和可维护性显著提升，也为今后的功能迁移和二次开发打下了良好基础。

在 FPGA 硬件设计中，面对高云 ACG720-138K 平台的资源优化与实时性挑战，我在集成五大功能模块（如逻辑分析仪、协议解析、信号发生器等）时多次面临资源超限风险。初期资源占用率超过 80%，导致时序难以收敛。通过重构模块复用架构，最终将资源占用率降至 20%，并保证了功能完整性。

时序收敛是另一大挑战。PWM 模块在仿真中波形完美，但实测存在周期性抖动。通过 SignalTap 分析发现时钟使能信号与数据路径存在 1.2ns 偏移。通过添加多周期约束并压缩关键路径逻辑级数，最终将时钟偏移控制在 200ps 以内。这一过程让我认识到实测验证的重要性。

值得强调的是测量模块的创新优化。针对传统频率计高频测量不稳定的问题，采用等精度测量法配合硬件智能触发：仅当信号跳变沿到达时激活测量计数器，避免高频时钟持续计数带来的功耗与误差。同时创新性地将量程提示整合到 LED 灯光中——LED 亮表示 ns 档，LED 不亮表示 us 档，既节省数码管资源又直观提升用户体验。

本次开发实践使我深刻认识到：硬件描述语言不仅是功能实现工具，更是物理电路的直接映射。每一行代码都对应着真实的门级电路，必须同时兼顾逻辑功能、时序约束和物理资源特性。从初期资源规划的不足到最终时序闭环的达成，这段经历重塑了我对硬件系统设计的认知——唯有将仿真、约束与实测三者紧密结合，才能打造真正可靠的硬件平台。

5 第五部分 参考文献

- [1] Ruoyuguize. ACG720 ACM2108 数据采集 DDR3 缓存 USB CDC 传输实验. 芯路恒电子. 2025 年 9 月. <https://fpga.cn/forum.php?mod=viewthread&tid=30175>
- [2] Ruoyuguize. 使用 CY7C68013 实现 USB-CDC 串口与 SPI 功能. 芯路恒电子. 2025 年 8 月. <https://fpga.cn/forum.php?mod=viewthread&tid=30150>
- [3] 野火 FPGA. FPGA Verilog 开放实战指南—基于 Intel Cyclone IV. 机械工业出版社.